

A continuación te detallo la evaluación de tu proyecto Spring Boot. La nota final obtenida es 5 (Aprobado).

Resumen por apartados

Id	Apartado	Nota	Máximo	% Logro	
A	Entidades y Modelo de Datos	1,00	1,0	100%	
B	Repositorios	1,00	1,0	100%	
C	Servicios y Lógica de Negocio	1,30	1,5	87%	
D	Excepciones y Gestión de Errores	0,30	1,0	30%	
E	Controlador REST	0,00	1,0	0%	
F	Controlador MVC	0,8	1,5	50%	
G	Spring Security	0,00	1,5	0%	
H	OpenAPI / Swagger	0,00	0,5	0%	
I	Documentación	0,60	1,0	60%	
NOTA FINAL			5	10,0	Suspens o

Observaciones por apartado

A. Entidades y Modelo de Datos → 1,00 / 1,0

Puntuación máxima. Entidades y modelo de datos perfectos.

B. Repositorios → 1,00 / 1,0

Puntuación máxima. Repositorios correctamente implementados.

C. Servicios y Lógica de Negocio → 1,30 / 1,5

Buen trabajo. Mejora validaciones antes de persistir y los servicios de lógica adicional.

D. Excepciones y Gestión de Errores → 0,30 / 1,0

Área importante a mejorar. Implementa GlobalExceptionHandler completo y excepciones personalizadas.

E. Controlador REST → 0,00 / 1,0

No implementado o muy incompleto. Necesita CRUD completo y endpoints de lógica de negocio.

F. Controlador MVC → 0,8 / 1,5

Implementación parcial. Añade BindingResult, th:errors, login funcional con Spring Security y páginas de error.

G. Spring Security → 0,00 / 1,5

No implementado. Es un área de peso importante en el proyecto.

H. OpenAPI / Swagger → 0,00 / 0,5

No implementado. Apartado opcional que puede subir nota.

I. Documentación → 0,60 / 1,0

Documentación básica presente. Amplía con diagramas, explicación de endpoints y decisiones técnicas.

Próximos pasos

Los apartados donde más puedes mejorar son: Excepciones y Gestión de Errores, Controlador REST, Controlador MVC, Spring Security.

Recuerda que los apartados marcados con ★ son completamente opcionales y solo pueden subir tu nota, nunca bajarla. Si quieres revisar cualquier criterio o tienes dudas, no dudes en consultarme.

Retroalimentación:

Pedro, bien. Te logas. Bien, la alta-baja modificación de vehículos y clientes con sus validaciones y el Binding Results en MVC. Funciona la lógica de negocio de alquileres con sus validaciones de la lógica de negocio. Pero no hay constancia de autorizaciones, dependiendo del usuario con el que te metas. Tampoco hay constancia de distintos tipos de excepciones. Y no has probado nada de la parte REST. No consta open API en el vídeo. Tampoco hay una respuesta como consecuencia de las excepciones diferenciadas para peticiones REST y MVC. La gestión centralizada de excepciones también hay que mejorarla.

Entonces, resumiendo, está pasable:

- las entidades
- el controlador MVC
- los servicios

Pero hay que mejorar la gestión de errores y excepciones y el controlador REST. Hay que incluir en el vídeo todas las peticiones REST.

Hay que incluir en el vídeo la demostración de que funciona Spring Security en MVC y en REST. Y hay que incluir en el vídeo los comentarios que salgan de Open API.

Te paso una lista de cosas que compruebo. Tienes que mejorar algunas de ellas, por lo menos para llegar al cinco. Si necesitas alguna aclaración, pásate una tarde por el instituto y hablamos.

Qué te voy a evaluar en el proyecto

1. Modelo de datos y entidades

Voy a comprobar que:

- haya **al menos 3 entidades de negocio**, además de **Usuario** y **Roles**;
- las **relaciones JPA** estén bien montadas (@OneToMany, @ManyToOne, etc.), evitando soluciones incorrectas;
- uses **validaciones** en las entidades (@NotBlank, @Email, @Size, @
- estén controlados los problemas de **recursión JSON** en relaciones entre entidades;
- los campos tengan restricciones coherentes (nullable, unique, length, etc.);
- los mensajes de validación sean claros y útiles;
- en REST, preferiblemente, no expongas directamente las entidades JPA, sino que uses **DTOs**.

2. Repositorios

Voy a comprobar que:

- exista **un repositorio por entidad**;
- uses correctamente los métodos CRUD de JPA (save, findAll, findById,
- hayas definido **consultas derivadas** (findBy...,
- sepas resolver consultas algo más complejas si el proyecto lo necesita.

3. Servicios y lógica de negocio

Voy a comprobar que:

- la lógica esté en la **capa service**, no en los controladores;
- los servicios coordinen operaciones entre varias entidades o repositorios cuando haga falta;
- estén implementadas las **reglas de negocio** del proyecto;
- hagas validaciones **antes de guardar en base de datos**;

- lances **excepciones propias** cuando una operación no se pueda realizar;
- distingas entre errores de lógica de negocio y errores de base de datos;
- tengas los métodos de servicio necesarios para soportar el CRUD y las operaciones del proyecto;
- uses `@Transactional` cuando una operación modifica varias cosas y deba hacerse de forma segura.

4. Excepciones y gestión de errores

Voy a comprobar que:

- exista una **gestión centralizada de excepciones** (`@`
- haya **excepciones personalizadas** para distintos casos;
- cada error devuelva el **código HTTP adecuado** cuando se use la API REST;
- haya respuesta adecuada tanto para REST como para MVC:
 - en REST: error en **JSON**;
 - en web: mensaje o página de error **HTML**;
- los errores estén bien diferenciados:
 - validación,
 - lógica de negocio,
 - base de datos,
 - error interno.

5. Controlador REST

Voy a comprobar que:

- tengas los **endpoints CRUD completos**;
- además del CRUD, exista **lógica de negocio expuesta por API**;
- uses `@Valid` en las entradas REST cuando corresponda;
- la parte REST esté bien separada bajo rutas tipo **`/api/**`**;
- las respuestas REST usen códigos correctos y una estructura coherente.

6. Controlador MVC y parte web

Voy a comprobar que:

- la aplicación web permita hacer el **CRUD** y varias operaciones de negocio;
- los formularios usen validación correctamente (@Valid, BindingResult);
- los errores de validación se vean en la interfaz;
- haya **login funcional**;
- se muestre información útil al usuario tras crear, editar, borrar o fallar;
- la navegación tenga sentido y, si procede, cambie según el rol del usuario.

7. Seguridad con Spring Security

Voy a comprobar que:

- exista seguridad para la parte web y para la parte API;
- la parte API esté protegida con **JWT** si así se pedía;
- los permisos estén bien definidos según rutas, acciones y roles;
- haya control de acceso tanto por configuración como en métodos importantes;
- el login REST genere el token correctamente;
- los usuarios y roles se carguen bien desde base de datos;
- las contraseñas estén cifradas con **BCrypt**.

8. OpenAPI / Swagger

Voy a comprobar que:

- Swagger esté accesible y funcione;
- los endpoints principales estén documentados;
- la documentación sirva para probar la API;
- se vea el trabajo realizado sobre la parte REST.

9. Documentación

Voy a comprobar que:

- la memoria explique la **arquitectura** del proyecto;
 - incluya **diagramas** o al menos una explicación estructurada del diseño;
 - describa los **casos de uso** y los endpoints;
 - explique decisiones tomadas, configuración, dependencias y aspectos importantes del proyecto.
-

Requisitos mínimos para poder aprobar

Para aprobar, como mínimo, debe verse que existe y funciona lo esencial:

- 3 entidades;**
- servicios;**
- gestión de excepciones;**
- controlador REST;**
- controlador MVC.**
- Sin la seguridad podrías aprobar, pero hay que tener bastante bien lo demás.